



was macht eine Favoritenseite für skincare Produkte in einer flask webanwendung

Eine Favoritenseite ist im Grunde die Seite, auf der ein eingeloggter Nutzer seine gemerkten Skincare-Produkte gesammelt sieht – technisch ist das aber ein Zusammenspiel aus Datenmodell, Routen und Templates.^[1] ^[2]

Kernaufgaben einer Favoritenseite

- Speichern, welche Produkte ein Nutzer favorisiert (z.B. in einer Tabelle favorites mit user_id und product_id oder in der Session, wenn du kein Login hast).^[3] ^[1]
- Anzeigen nur der Favoriten des aktuell eingeloggten Nutzers, nicht aller Nutzer.^[1]
- Buttons oder Icons (Herz/Stern) an jedem Skincare-Produkt, um Favoriten hinzuzufügen oder zu entfernen (POST-Route wie /add_favorite / remove_favorite).^[2] ^[1]
- Eine Route wie /favorites, die die Favoriten des aktuellen Nutzers lädt (Query filter(Favorite.user_id == current_user.id)) und an ein Template übergibt.^[1]
- Im Template die Liste rendern, z.B. Karten mit Produktbild, Name, Hauttyp, Link zur Produktdetailseite usw., angepasst an Skincare-Kontext.^[4] ^[5]

Beispiel grob in Flask gedacht

- Datenmodell: User, Product, Favorite (Many-to-Many zwischen User und Product).^[2] ^[1]
- Route:
 - `@app.route("/favorites")`: hole favorites = `Favorite.query.filter_by(user_id=current_user.id).all()` und rendere favorites.html.^[1]
 - `@app.route("/products/<int:id>/favorite", methods=["POST"])`: fügt ein Produkt zu den Favoriten des Nutzers hinzu.^[2] ^[1]
- Template favorites.html: Schleife über die Favoriten und Darstellung der Produkte (Bild, Name, evtl. Hauttyp-Filter, Preis).^[5] ^[4] ^[1]

Wenn du magst, kann ich dir im nächsten Schritt ein kleines konkretes Flask-Codebeispiel (Model, Route, Template-Snippet) für eine Skincare-Favoritenseite schreiben.

ja bitte

Hier ein minimales, aber vollständiges Beispiel für eine Favoriten-Funktion in einer Flask-App für Skincare-Produkte.

Models (User, Product, Favorite)

```
from flask_sqlalchemy import SQLAlchemy
from flask_login import UserMixin

db = SQLAlchemy()

# Many-to-Many Zwischentabelle
favorites = db.Table(
    "favorites",
    db.Column("user_id", db.Integer, db.ForeignKey("user.id"), primary_key=True),
    db.Column("product_id", db.Integer, db.ForeignKey("product.id"), primary_key=True),
)

class User(db.Model, UserMixin):
    id = db.Column(db.Integer, primary_key=True)
    email = db.Column(db.String(120), unique=True, nullable=False)
    # Beziehung: welche Produkte dieser User favorisiert
    favorite_products = db.relationship(
        "Product",
        secondary=favorites,
        backref="favorited_by",
        lazy="dynamic",
    )

class Product(db.Model):
    id = db.Column(db.Integer, primary_key=True)
    name = db.Column(db.String(120), nullable=False)
    brand = db.Column(db.String(120))
    skin_type = db.Column(db.String(50)) # z.B. "dry", "oily", "sensitive"
    price = db.Column(db.Float)
```

Die Many-to-Many-Beziehung über eine Zwischentabelle ist ein übliches Muster für Favoritenfunktionen. [\[11\]](#) [\[12\]](#)

Routen zum Hinzufügen/Entfernen

Hier gehe ich davon aus, dass du flask-login verwendest und current_user verfügbar ist.

```
from flask import Flask, render_template, redirect, url_for, request, flash
from flask_login import current_user, login_required

app = Flask(__name__)

@app.route("/products")
def product_list():
    products = Product.query.all()
    return render_template("products.html", products=products)
```

```

@app.route("/favorites")
@login_required
def favorites_page():
    # alle Favoriten des eingeloggten Users
    favorites = current_user.favorite_products.all()
    return render_template("favorites.html", products=favorites)

@app.route("/products/<int:product_id>/toggle_favorite", methods=["POST"])
@login_required
def toggle_favorite(product_id):
    product = Product.query.get_or_404(product_id)

    if product in current_user.favorite_products:
        # entfernen
        current_user.favorite_products.remove(product)
        flash("Aus Favoriten entfernt.")
    else:
        # hinzufügen
        current_user.favorite_products.append(product)
        flash("Zu Favoriten hinzugefügt.")

    db.session.commit()
    # zurück zur Seite, von der der User gekommen ist
    next_url = request.referrer or url_for("product_list")
    return redirect(next_url)

```

Die Beziehung `favorite_products` erlaubt es, direkt `append` und `remove` auf der Liste zu verwenden. [\[12\]](#) [\[11\]](#)

Template: Produktübersicht mit Herz-Button

templates/products.html:

```

<!doctype html>
<html>
<head>
    <title>Skincare Produkte</title>
</head>
<body>
    <h1>Skincare Produkte</h1>

    {% for product in products %}
        <div>
            <h3>{{ product.name }} - {{ product.brand }}</h3>
            <p>Hauttyp: {{ product.skin_type }}</p>
            <p>Preis: {{ "%.2f"|format(product.price) }} €</p>

            {% if current_user.is_authenticated %}
                <form action="{{ url_for('toggle_favorite', product_id=product.id) }}" method="post">
                    {% if product in current_user.favorite_products %}
                        <button type="submit">♥ Entfernen</button>
                    {% else %}
                        <button type="submit">♥ Favorit</button>

```

```

        {% endif %}
    </form>
    {% endif %}
</div>
<hr>
{% endfor %}
</body>
</html>

```

Jinja-`for`-Schleifen sind der Standardweg, um Listen (wie Produkte oder Favoriten) im Template auszugeben. [\[13\]](#) [\[14\]](#)

Template: Favoritenseite

templates/favorites.html:

```

<!doctype html>
<html>
<head>
    <title>Meine Favoriten</title>
</head>
<body>
    <h1>Meine Skincare-Favoriten</h1>

    {% if products %}
        {% for product in products %}
            <div>
                <h3>{{ product.name }} - {{ product.brand }}</h3>
                <p>Hauttyp: {{ product.skin_type }}</p>
                <p>Preis: {{ "%.2f"|format(product.price) }} €</p>

                <form action="{{ url_for('toggle_favorite', product_id=product.id) }}" method="post">
                    <button type="submit">Aus Favoriten entfernen</button>
                </form>
            </div>
            <hr>
        {% endfor %}
    {% else %}
        <p>Du hast noch keine Favoriten gespeichert.</p>
    {% endif %}
</body>
</html>

```

So entsteht eine einfache Favoritenseite, auf der ein Nutzer nur seine eigenen gemerkten Skincare-Produkte sieht und direkt verwalten kann. [\[11\]](#) [\[12\]](#)

Wenn du mir sagst, ob du bereits Login/Registrierung und Datenbank eingerichtet hast, kann ich das Beispiel genau an deine bestehende Struktur (z.B. Blueprints, Config, Models) anpassen.



Erkläre mir was diese Favoriten Seite macht (Frontend und Backend)

Die Favoritenseite ist das Zusammenspiel aus zwei Teilen: Im **Backend** verwaltest du Daten (welcher User welche Produkte mag), im **Frontend** zeigst du diese Daten an und schickst Aktionen (Favorit an/aus) zurück ans Backend. [\[21\]](#) [\[22\]](#) [\[23\]](#)

Backend: Daten und Logik

- Datenmodell:
 - Es gibt `User` und `Product`.
 - Dazwischen liegt eine Many-to-Many-Beziehung `favorites` (Zuordnungstabelle mit `user_id` und `product_id`). So kann ein User viele Produkte favorisieren und ein Produkt von vielen Usern favorisiert werden. [\[22\]](#) [\[24\]](#)
- Beziehung im Code:
 - Beim User gibt es ein Attribut `favorite_products = db.relationship("Product", secondary=favorites, ...)`.
 - Dadurch kannst du im Code Dinge tun wie `current_user.favorite_products.append(product)` oder `remove(product)`, um Favoriten zu speichern oder zu löschen. [\[25\]](#) [\[22\]](#)
- Routen:
 - `/favorites` (GET):
 - Holt alle Produkte, die der aktuell eingeloggte User favorisiert, z.B. `current_user.favorite_products.all()`.
 - Übergibt diese Liste an das Template `favorites.html`, das dann die Favoritenseite darstellt. [\[23\]](#) [\[21\]](#)
 - `/products/<int:product_id>/toggle_favorite` (POST):
 - Nimmt aus der URL die `product_id`.
 - Sucht das Produkt in der Datenbank.
 - Prüft, ob das Produkt schon in `current_user.favorite_products` ist:
 - Falls ja: `remove(product)` → Favorit wird gelöscht.
 - Falls nein: `append(product)` → Favorit wird gespeichert.
 - Speichert die Änderung mit `db.session.commit()` und leitet den User wieder zurück (z.B. zur Produktliste). [\[22\]](#) [\[23\]](#)

Damit kümmert sich das Backend darum, welche Einträge in der Favoriten-Beziehungstabelle stehen, also welche Skincare-Produkte zu welchem Nutzer gehören. [\[21\]](#) [\[22\]](#)

Frontend: Templates, Buttons und Anzeige

- Produktliste (`products.html`):
 - Das Backend übergibt eine Liste von `products` an das Template.
 - Im Template läuft eine Jinja-Schleife:
 - Für jedes Produkt werden Name, Marke, Hauttyp, Preis usw. angezeigt.
 - Wenn ein Nutzer eingeloggt ist, wird ein Formular mit einem Button angezeigt, das auf `/products/<product_id>/toggle_favorite` zeigt. [\[26\]](#) [\[23\]](#)
 - Der Button-Text/Icon hängt davon ab, ob dieses Produkt schon in den Favoriten des Users ist:
 - Wenn ja: z.B. „♥ Entfernen“.
 - Wenn nein: z.B. „🔖 Favorit“. [\[23\]](#) [\[26\]](#)
- Formular/Request:
 - Das Formular im Template hat `method="post"` und `action="{{ url_for('toggle_favorite', product_id=product.id) }}"`.
 - Beim Klick auf den Button sendet der Browser einen POST-Request an die Toggle-Route im Backend, ohne dass du JavaScript brauchst. [\[27\]](#) [\[23\]](#)
- Favoritenseite (`favorites.html`):
 - Das Backend übergibt hier nicht alle Produkte, sondern nur `current_user.favorite_products.all()` als `products`.
 - Das Template zeigt ähnlich wie `products.html` die Produktinformationen an, aber nur für diese Favoriten.
 - Es gibt wieder Buttons/Formulare, um ein Produkt aus den Favoriten zu löschen (ruft intern die gleiche Toggle-Route auf). [\[26\]](#) [\[21\]](#) [\[23\]](#)

Damit macht die Favoritenseite im Frontend: sie listet nur die Skincare-Produkte, die der aktuelle Nutzer zuvor als Favorit markiert hat, und bietet Buttons zum Entfernen. Im Backend sorgt die Many-to-Many-Beziehung und die Toggle-Route dafür, dass diese Liste immer zur User-Aktion passt (hinzufügen/entfernen und Speichern in der Datenbank). [\[21\]](#) [\[22\]](#) [\[23\]](#) [\[26\]](#)



Welche Daten fließen hier von der Datenbank bis zur HTML-Seite? Bitte als Ablauf

Von der Datenbank bis zur HTML-Seite kannst du dir den Ablauf wie eine Kette von Objekten/Datenstrukturen vorstellen, die nacheinander entstehen. [\[32\]](#) [\[33\]](#)

1. Daten in der Datenbank

- Tabellen: `user`, `product` und die Verknüpfungstabelle `favorites` mit `user_id` und `product_id`.
- Inhalt: In `product` stehen z.B. `id`, `name`, `brand`, `skin_type`, `price`; in `favorites` steht, welcher User welches Produkt favorisiert. [\[34\]](#) [\[33\]](#)

2. ORM-Objekte in Python (Backend)

- Flask-SQLAlchemy liest beim Aufruf der Route (z.B. `/favorites`) die passenden Datensätze aus der Datenbank.
- Die Query `current_user.favorite_products.all()` holt alle Produkte, deren `id` in der `favorites`-Tabelle mit der `user_id` des eingeloggten Users verknüpft ist, und wandelt sie in Python-Objekte der Klasse `Product` um. [\[33\]](#) [\[34\]](#)

3. Kontext für das Template

- In der View-Funktion machst du z.B.:

```
return render_template("favorites.html", products=favorites)
```
- Hier wird eine Python-Liste von `Product`-Objekten (`favorites`) in den Template-Kontext gepackt und unter dem Namen `products` an das Template übergeben. [\[35\]](#) [\[36\]](#)

4. Verarbeitung im Template (Jinja2)

- Im Template `favorites.html` iterierst du über `products`:

```
{% for product in products %}
```
- Innerhalb der Schleife greifst du auf die Attribute der ORM-Objekte zu: `product.name`, `product.brand`, `product.skin_type`, `product.price`.
- Zusätzlich steht im Kontext `current_user`, über den das Template z.B. prüfen kann, ob der Nutzer eingeloggt ist oder ob ein Produkt in seinen Favoriten ist. [\[37\]](#) [\[38\]](#)

5. Generiertes HTML

- Jinja2 ersetzt alle Platzhalter (`{{ ... }}`) und Kontrollstrukturen (`{% ... %}`) durch reinen HTML-Text.
- Ergebnis ist eine fertige HTML-Seite, in der für jedes favorisierte Produkt die konkreten Werte eingefügt sind (Produktname, Marke, Hauttyp, Preis, Button-Formular mit richtiger `product_id` in der URL). [\[36\]](#) [\[37\]](#)

6. Antwort an den Browser

- Flask schickt dieses fertige HTML als HTTP-Response an den Browser.
- Der Browser rendert die Seite und zeigt dem User seine Favoritenliste samt Buttons/Formularen an; beim Klick auf einen Button sendet der Browser dann wieder eine neue Anfrage (POST auf die Toggle-Route), womit der Zyklus neu startet. [\[35\]](#) [\[36\]](#)

Kurz: Datenbank-Zeile → ORM-Objekt (Product) → Liste products im Template-Kontext → Jinja2 erzeugt HTML mit konkreten Werten → Browser zeigt die Favoritenseite an. [\[32\]](#) [\[36\]](#)

✱

Welche Routen/Endpoints sind beteiligt und wofür ist jede Route zuständig?

Beteiligt sind typischerweise drei Haupt-Routen rund um die Favoritenfunktion; dazu kommen meist allgemeine Routen wie Login oder Produktliste. [\[45\]](#) [\[46\]](#)

1. Produktlisten-Route (GET /products)

- Zuständig dafür, alle (oder gefilterte) Skincare-Produkte aus der Datenbank zu laden.
- Erstellt daraus eine Python-Liste von Product-Objekten und übergibt sie an das Template products.html (z.B. `render_template("products.html", products=products)`).
- Im Template werden die Produkte angezeigt, inklusive „Favorit“-Buttons, die wiederum auf die Toggle-Route verweisen. [\[46\]](#) [\[45\]](#)

2. Favoritenseiten-Route (GET /favorites)

- Lädt nur die Produkte, die der aktuell eingeloggte Nutzer favorisiert hat, z.B. `current_user.favorite_products.all()`.
- Übergibt diese Liste an favorites.html, sodass dort nur die Favoriten angezeigt werden.
- Diese Route ist die eigentliche „Favoritenseite“, also die Übersicht für den User. [\[47\]](#) [\[45\]](#)

3. Toggle-Favorit-Route (POST /products/<int:product_id>/toggle_favorite)

- Wird vom Formular/Button im Frontend aufgerufen, wenn der User ein Produkt als Favorit markieren oder wieder entfernen möchte.
- Nimmt die product_id aus der URL, sucht das passende Produkt in der Datenbank und prüft, ob es bereits in `current_user.favorite_products` liegt.
- Wenn ja: Produkt wird aus der Favoriten-Beziehung entfernt; wenn nein: es wird hinzugefügt; danach wird gespeichert und der User wieder zurück auf die vorige Seite (z.B. Produktliste oder Favoritenseite) umgeleitet. [\[48\]](#) [\[49\]](#)

4. (Indirekt wichtig) Auth-Routen wie Login (GET/POST /login)

- Nicht Teil der Favoritenlogik selbst, aber notwendig, damit `current_user` existiert und die Favoriten pro Nutzer gespeichert werden können.
- Ohne diese Routen gäbe es entweder keine Nutzertrennung oder du müsstest Favoriten nur in der Session speichern statt in der User-Beziehung. [\[50\]](#) [\[51\]](#)

Kurz: Die Produktlisten-Route zeigt alle Produkte, die Favoritenseiten-Route zeigt nur die gemerkten, und die Toggle-Route verknüpft/entkoppelt im Hintergrund Nutzer und Produkt in der Datenbank, ausgelöst durch die Buttons im Frontend. [\[45\]](#) [\[46\]](#)



Was bedeutet `{% for p in products %}` in Jinja und woher kommt `products`?

`{% for p in products %}` ist in Jinja eine Schleife, mit der du über alle Elemente in einer Sammlung iterierst, ganz ähnlich wie `for p in products:` in Python. [\[58\]](#) [\[59\]](#)

Bedeutung von `{% for p in products %}`

- `{% ... %}` kennzeichnet einen Steuerbefehl (kein sichtbarer Text im HTML, sondern Template-Logik).
- `for` ist die Jinja-Schleife.
- `p` ist eine Schleifenvariable: In jedem Durchlauf steht `p` für ein einzelnes Element der Sammlung.
- `products` ist die Sammlung (z.B. Liste von Produkt-Objekten), über die iteriert wird.
- Die Schleife endet später mit `{% endfor %}`; alles dazwischen wird für jedes Element einmal ausgeführt. [\[59\]](#) [\[58\]](#)

Woher kommt `products`?

`products` entsteht im Python-Code deiner Flask-Route, bevor `render_template` aufgerufen wird:

```
@app.route("/favorites")
@login_required
def favorites_page():
    favorites = current_user.favorite_products.all() # Python-Liste/Query-Ergebnis
    return render_template("favorites.html", products=favorites)
```

- `favorites` ist hier eine Liste von `Product`-Objekten aus der Datenbank.
- Mit `render_template("favorites.html", products=favorites)` übergibst du diese Liste in den Template-Kontext unter dem Namen `products`.
- Im Template kannst du dann genau diesen Namen verwenden: `products`. Daher funktioniert `{% for p in products %}` überhaupt – es greift auf die Variable zu, die du aus der Route hinein gegeben hast. [\[60\]](#) [\[61\]](#)

Kurz: `products` kommt immer aus der Route, weil du es dort an `render_template(..., products=...)` übergibst; Jinja nutzt das dann in der Schleife `{% for p in products %}`, um für jedes Produkt HTML zu erzeugen.



Erkläre mir die Struktur der Seite: header, grid, card, also was wird wo gerendert?

Die Seite besteht grob aus einem statischen Rahmen (z.B. Header/Layout) und einem dynamischen Bereich, in dem die Produkt-Karten im Grid gerendert werden. [\[70\]](#) [\[71\]](#)

Header / Grundlayout

- Meist gibt es eine Basis-Vorlage, z.B. `base.html`, mit HTML-Grundstruktur (`<html>`, `<head>

- Deine Favoritenseite erweitert diese Basis mit `{% extends "base.html" %}` und füllt innerhalb eines `{% block content %}...{% endblock %}` den eigentlichen Seiteninhalt.

- In diesem Block steht dann z.B. eine Überschrift wie `<h1>Meine Skincare-Favoriten</h1>`

Grid-Bereich (Produkt-Anordnung)

- Unterhalb des Headers/der Überschrift legst du ein Grid oder eine Zeile an, z.B. mit CSS

- In diesem Grid-Container laufen dann die Jinja-Schleifen: `{% for p in products %}` erzeugt für jedes Produkt eine neue Zelle/Spalte im Grid.
- Das Grid-Layout selbst (wie viele Spalten, Abstände usw.) wird über CSS oder ein Framework wie Bootstrap gesteuert, nicht über Jinja. [\[72\]](#) [\[73\]](#)

Card-Komponenten (pro Produkt)

- Innerhalb der Schleife `{% for p in products %}` wird eine „Card“ pro Produkt gebaut, z.B.:

```
<div class="card">
  <h3>{{ p.name }} - {{ p.brand }}</h3>
  <p>Hauttyp: {{ p.skin_type }}</p>
  <p>Preis: {{ "%.2f"|format(p.price) }} €</p>
  <form action="{{ url_for('toggle_favorite', product_id=p.id) }}" method="post">
    <button type="submit">Aus Favoriten entfernen</button>
  </form>
</div>
```

- Die Card ist ein normales `<div>` mit einer Card-CSS-Klasse; darin werden die dynamischen Daten (name, brand, skin_type, price) via `{{ ... }}` eingefügt.
- Der Button/Formular-Teil in der Card schickt bei Klick einen POST-Request an den Favoriten-Toggle-Endpoint, mit der passenden `product_id` in der URL. [\[73\]](#) [\[70\]](#)

Was wird wo gerendert?

- Header / Navigation / allgemeines Layout:
 - In der Basis-Vorlage (z.B. `base.html`) oder in eingebundenen Teil-Templates (`{% include "navbar.html" %}`).
 - Wird bei jeder Seite gleich angezeigt und enthält keine oder nur wenige dynamische Daten. [\[74\]](#) [\[70\]](#)
- Seiten-Content (Titel + Grid):
 - In der kindlichen Template-Datei (z.B. `favorites.html`) innerhalb von `{% block content %}`.
 - Hier wird die Überschrift, optional Filter/Info-Text und dann das Grid-Container-Element gerendert. [\[70\]](#) [\[75\]](#)
- Cards (pro Produkt im Grid):
 - Innerhalb des Grids in der Schleife `{% for p in products %}`.
 - Jedes Durchlaufen der Schleife erzeugt eine Card mit den Daten eines Produkts und integriert den Button für die Favoritenaktion. [\[72\]](#) [\[73\]](#)

Wenn du möchtest, kann ich dir ein konkretes, vollständiges `favorites.html` mit Header, Grid und Cards schreiben, das zu deinem bisherigen Python-Code passt.

✱

**so?:<section class="grid"> {% for p in products %}
%} <article class="card">**

Ja, genau! Das ist die perfekte Struktur für die Produktliste oder Favoritenseite.

```
<section class="grid">
  {% for p in products %}
    <article class="card">
      <!-- Card-Inhalt hier -->
    </article>
  {% endfor %}
</section>
```

Erklärung der Struktur

`<section class="grid">`

- Container für das gesamte Grid-Layout
- CSS Grid oder Flexbox sorgt dafür, dass die Cards automatisch nebeneinander angeordnet werden
- Gerendert **einmal** außerhalb der Schleife vom Backend

`{% for p in products %}`

- Jinja-Schleife: für jedes Produkt in der `products`-Liste (aus der Route)
- Gerendert **dynamisch** so oft wie Produkte vorhanden sind

```
<article class="card">
```

- Einzelne Produktkarte (semantisch korrekt mit `<article>`)
- CSS `.card` Klasse für Hover-Effekte, Schatten, abgerundete Ecken etc.
- Gerendert **pro Produkt** innerhalb der Schleife

Vollständiges Beispiel `favorites.html`:

```
{% extends "base.html" %}
{% block content %}
    <h1>Meine Favoriten</h1>

    <section class="grid">
        {% for p in products %}
            <article class="card">
                <h3>{{ p.name }} - {{ p.brand }}</h3>
                <p>Hauttyp: {{ p.skin_type }}</p>
                <p>{{ "%.2f"|format(p.price) }} €</p>
                <form action="{{ url_for('toggle_favorite', product_id=p.id) }}" method="post">
                    <button>Aus Favoriten entfernen</button>
                </form>
            </article>
        {% endfor %}
    </section>
{% endblock %}
```

Was wird wo gerendert:

- `<section class="grid">` → **Einmal** (statisch)
- `<article class="card">` → **N-mal** (dynamisch, N = Anzahl Favoriten)
- Alle `{{ p.name }}` etc. → **Pro Card** mit echten Produktdaten

Die CSS-Klassen `grid` und `card` machst du dann in deiner `style.css` schön responsive. Perfekt strukturiert! ☐

*
**

Was macht der `{% else %}`-Teil in der `for`-Schleife genau?

Der `{% else %}`-Teil in der **for-Schleife** wird **nur ausgeführt, wenn die Liste `products` leer ist** (also keine Schleifendurchläufe stattfinden). [\[89\]](#) [\[90\]](#)

Vollständiges Beispiel:

```
<section class="grid">
  {% for p in products %}
    <article class="card">
      <h3>{{ p.name }}</h3>
      <!-- Produktinhalt -->
    </article>
  {% else %}
    <p>Du hast noch keine Favoriten!</p>
  {% endfor %}
</section>
```

Was passiert genau:

Fall 1: products hat Elemente (z.B. 3 Favoriten)

```
products = [produkt1, produkt2, produkt3]
```

→ Schleife läuft **3x** durch → **3 Cards** werden gerendert → `{% else %}` wird **ignoriert**

Fall 2: products ist leer []

```
products = []
```

→ Schleife läuft **0x** durch → `{% else %}` wird **ausgeführt** → `<p>Du hast noch keine Favoriten!</p>` erscheint

Praktischer Ablauf in deiner Route:

```
@app.route("/favorites")
def favorites():
    products = current_user.favorite_products.all() # Könnte leer sein!
    return render_template("favorites.html", products=products)
```

Backend übergibt:

- User hat Favoriten → `products = [obj1, obj2]` → Grid mit Cards
- User hat keine Favoriten → `products = []` → "Keine Favoriten"-Meldung

Das ist super nützlich für UX: Statt einem leeren Grid siehst du eine hilfreiche Nachricht! ☺



welche werte werden von p benutzt und was müssen das für Spalten/Felder in der DB sein?

In `p` (der Schleifenvariable `{% for p in products %}`) stehen folgende Werte zur Verfügung, die direkt aus dem Product-Modell kommen:

Verwendete `p`-Attribute im Template

```
{{ p.name }}      → Produktname
{{ p.brand }}     → Marke
{{ p.skin_type }} → Hauttyp
{{ p.price }}     → Preis
{{ p.id }}        → Primärschlüssel (für Form-URL)
```

Entsprechende DB-Felder im Product-Modell

```
class Product(db.Model):
    id = db.Column(db.Integer, primary_key=True)      # INT PRIMARY KEY
    name = db.Column(db.String(120), nullable=False)  # VARCHAR(120)
    brand = db.Column(db.String(120))                 # VARCHAR(120)
    skin_type = db.Column(db.String(50))              # VARCHAR(50)
    price = db.Column(db.Float)                      # DECIMAL(10,2) oder FLOAT
```

Datenbank-Tabelle `product`

Spalte	Datentyp	Beispielwert
id	INTEGER PK	1
name	VARCHAR(120)	"Hydra Boost Serum"
brand	VARCHAR(120)	"The Ordinary"
skin_type	VARCHAR(50)	"dry"
price	DECIMAL(10,2)	24.99

Zusammenhang: Jinja greift über `p.name` auf `product.name` (DB-Spalte) zu. Das ORM (SQLAlchemy) macht aus DB-Zeilen automatisch Python-Objekte, deren Attribute direkt im Template verfügbar sind.

Müssen da sein: Mindestens `id`, `name` für grundlegende Funktionalität. `brand`, `skin_type`, `price` sind Skincare-spezifisch sinnvoll.

✱

```
# Ich hab's gerade so: <article class="card"> <a href="{{ url_for('product_details', prod
```

Perfekt! Das ist eine super Ergänzung für die Card-Struktur.

```
<article class="card">
  <a href="{{ url_for('product_details', product_id=p.id) }}">
    <!-- Card-Inhalt: Bild, Name, Preis etc. -->
  </a>
</article>
```

Was passiert hier genau:

```
{{ url_for('product_details', product_id=p.id) }}
```

- Generiert automatisch die URL zur Detailseite eines Produkts
- `product_details` = Name der Python-Funktion deiner Detail-Route
- `product_id=p.id` = Parameter, der in die URL eingefügt wird

Deine Route muss so aussehen:

```
@app.route('/product/<int:product_id>')
def product_details(product_id):
    product = Product.query.get_or_404(product_id)
    return render_template('product_details.html', product=product)
```

Ergebnis-URLs:

```
Produkt ID 1 → /product/1
Produkt ID 42 → /product/42
Produkt ID 1337 → /product/1337
```

Vollständige Card-Struktur:

```
<article class="card">
  <a href="{{ url_for('product_details', product_id=p.id) }}">
    <h3>{{ p.name }} - {{ p.brand }}</h3>
    <p>Hauttyp: {{ p.skin_type }}</p>
    <p>{{ "%.2f"|format(p.price) }} €</p>
  </a>
  <!-- Favoriten-Button AUSSERHALB des Links -->
  <form action="{{ url_for('toggle_favorite', product_id=p.id) }}" method="post">
    <button>Aus Favoriten entfernen</button>
  </form>
</article>
```

Wichtig: Der Favoriten-Button steht **außerhalb** des `<a>`-Tags, damit Klick auf Button ≠ Klick auf Link.

Vorteil: User kann direkt von der Favoritenübersicht zur Detailseite klicken, ohne den Favoriten-Status zu verlieren! ☐

Ich will einen getrennten Layout wie mach ich das?

Perfekt! Du willst getrennte Layouts für verschiedene Seiten. Das machst du mit **Template Inheritance** über `base.html` + spezifische Child-Templates.

1. Basis-Layout `templates/base.html` (allgemein)

```
<!DOCTYPE html>
<html>
<head>
    <title>{% block title %}Skincare Shop{% endblock %}</title>
    <link rel="stylesheet" href="{% url_for('static', filename='style.css') %}">
    {% block head %}{% endblock %}
</head>
<body>
    <nav>
        <a href="{% url_for('product_list') %}">Produkte</a>
        <a href="{% url_for('favorites') %}">Favoriten</a>
    </nav>

    <main>
        {% block content %}{% endblock %}
    </main>
</body>
</html>
```

2. Favoriten-spezifisches Layout `templates/favorites.html`

```
{% extends "base.html" %}
{% block title %}Meine Favoriten{% endblock %}

{% block content %}
    <header class="page-header">
        <h1>♥ Meine Skincare-Favoriten</h1>
    </header>

    <section class="grid">
        {% for p in products %}
            <article class="card">
                <a href="{% url_for('product_details', product_id=p.id) %}">
                    <h3>{{ p.name }} - {{ p.brand }}</h3>
                    <p>{{ p.skin_type }} Hauttyp</p>
                    <p>{{ "%.2f"|format(p.price) }} €</p>
                </a>
                <form action="{% url_for('toggle_favorite', product_id=p.id) %}" method='
                    <button>✕ Entfernen</button>
                </form>
            </article>
        {% else %}
            <p>Keine Favoriten gefunden. <a href="{% url_for('product_list') %}">Zu den F
        {% endfor %}
```



```
</section>
{% endblock %}
```

3. Produktliste mit anderem Layout `templates/products.html`

```
{% extends "base.html" %}
{% block title %}Alle Skincare Produkte{% endblock %}

{% block content %}
    <header class="page-header">
        <h1> Alle Produkte</h1>
        <div class="filters">
            <select>
                <option>Trockene Haut</option>
                <option>Fettige Haut</option>
            </select>
        </div>
    </header>

    <section class="grid">
        {% for p in products %}
            <article class="card">
                <!-- gleiche Card-Struktur, aber anderer Header -->
            </article>
        {% endfor %}
    </section>
{% endblock %}
```

4. Detailseite mit komplett anderem Layout `templates/product_details.html`

```
{% extends "base.html" %}
{% block content %}
    <div class="detail-layout">
        <aside class="product-image"><!-- Großes Bild --></aside>
        <article class="product-info">
            <h1>{{ product.name }}</h1>
            <!-- Detail-Inhalte -->
        </article>
    </div>
{% endblock %}
```

Wie funktioniert's?

- `base.html` = gemeinsame Struktur (Nav, CSS, HTML-Rahmen)
- `{% extends "base.html" %}` = erbt alles von `base.html`
- `{% block content %}` = überschreibt nur den Hauptinhalt
- **Jede Seite** hat ihr eigenes `header`, `grid`-Layout, Styling

Routes bleiben gleich:

```
@app.route("/favorites")
return render_template("favorites.html", products=favorites)

@app.route("/products")
return render_template("products.html", products=products)
```

So hast du **getrennte Layouts pro Seite**, aber **DRY-Prinzip** (Don't Repeat Yourself) durch gemeinsame Basis! ☐

✱

```
# gehts so?: <article class="card"> <div class="thumbWrap"> <a href="{{ url_for('product
```

Ja, absolut perfekt! Das ist eine richtig gute Card-Struktur für Produkte mit Bildern.

```
<article class="card">
  <div class="thumbWrap">
    <a href="{{ url_for('product_details', product_id=p.id) }}">
      <!-- Thumbnail/Bild hier -->
      
  </div>
  <!-- Rest der Card-Inhalte außerhalb -->
  <h3>{{ p.name }} - {{ p.brand }}</h3>
  <p>{{ p.skin_type }} Hauttyp</p>
  <p>{{ "%.2f"|format(p.price) }} €</p>
</article>
```

Warum das super ist:

```
<div class="thumbWrap">
```

- Container speziell für das **Thumbnail/Bild**
- Perfekt für CSS: fixed height, object-fit, Hover-Effekte, Loading-States
- Hält das Bild schön im Layout, ohne die Card zu sprengen

```
<a> um das Bild
```

- Klick auf Bild → Detailseite (intuitive UX)
- Bild + Link sind semantisch korrekt zusammengefasst

Vollständige Struktur:

```
<article class="card">
  <div class="thumbWrap">
    <a href="{{ url_for('product_details', product_id=p.id) }}">
      
```

```

    </a>
</div>

<div class="card-body">
  <h3>{{ p.name }}</h3>
  <p class="brand">{{ p.brand }}</p>
  <p class="skin-type">{{ p.skin_type }}</p>
  <p class="price">{{ "%.2f"|format(p.price) }} €</p>

  <!-- Favoriten Button -->
  <form action="{{ url_for('toggle_favorite', product_id=p.id) }}" method="post">
    <button>♥</button>
  </form>
</div>
</article>

```

CSS-Beispiel für thumbWrap:

```

.thumbWrap {
  height: 200px;
  overflow: hidden;
  border-radius: 8px 8px 0 0;
}

.thumbWrap img {
  width: 100%;
  height: 100%;
  object-fit: cover;
  transition: transform 0.3s;
}

.thumbWrap:hover img {
  transform: scale(1.05);
}

```

Dein Ansatz ist top! Der `thumbWrap` macht die Cards professionell und responsive. 🙌



herz button als `<form method="post">` oder als ein Link?

`<form method="post">` ist die richtige Wahl für den Herz-Button! 🙌

Warum `<form>` besser ist als `<a>` Link:

✓ <form method="post"> (empfohlen)

```
<form action="{{ url_for('toggle_favorite', product_id=p.id) }}" method="post">
  <button type="submit" class="heart-btn">♥</button>
</form>
```

Vorteile:

- **Datenbank-Änderung:** POST ist für Mutationen (Hinzufügen/Entfernen Favorit)
- **CSRF-Schutz:** Flask-WTForms unterstützt das automatisch
- **Toggle-Logik:** Prüft im Backend, ob Favorit schon existiert
- **Flash-Nachrichten:** flash("Zu Favoriten hinzugefügt!")
- **Semantisch korrekt:** Buttons für Aktionen, Links für Navigation

✗ <a> Link (nicht empfohlen)

```
<a href="{{ url_for('toggle_favorite', product_id=p.id) }}">♥</a>
```

Nachteile:

- **GET statt POST:** Browser-Back/Refresh löscht Favoriten wieder
- **Bookmarkbar:** Links werden gespeichert/gecached → Probleme
- **SEO/Crawler:** Suchmaschinen indexieren deine Favoriten-Änderungen
- **Keine Toggle-Logik:** Schwieriger zu prüfen "schon Favorit oder nicht"

Backend für Form (wie gehabt):

```
@app.route("/products/<int:product_id>/toggle_favorite", methods=["POST"])
@login_required
def toggle_favorite(product_id):
    product = Product.query.get_or_404(product_id)
    if product in current_user.favorite_products:
        current_user.favorite_products.remove(product)
        flash("Aus Favoriten entfernt")
    else:
        current_user.favorite_products.append(product)
        flash("Zu Favoriten hinzugefügt")
    db.session.commit()
    return redirect(request.referrer)
```

Vollständige Card:

```
<article class="card">
  <div class="thumbWrap">
    <a href="{{ url_for('product_details', product_id=p.id) }}"></a>
  </div>
  <div class="card-body">
    <h3>{{ p.name }}</h3>
    <p>{{ p.price }} €</p>

    <!-- Herz-Button als FORM -->
    <form action="{{ url_for('toggle_favorite', product_id=p.id) }}" method="post" class=
      {% if product in current_user.favorite_products %}
        <button type="submit" class="heart-btn active">♥</button>
      {% else %}
        <button type="submit" class="heart-btn"></button>
      {% endif %}
    </form>
  </div>
</article>
```

Fazit: Form = korrekt für Favoriten-Buttons! Links nur für Navigation zur Detailseite. ☐

✱

okay. geht das so?:♥

Ja, das geht perfekt! ☐

```
<form action="{{ url_for('toggle_favorite', product_id=p.id) }}" method="post">
  <button class="heart" type="submit" title="Favorit entfernen">♥</button>
</form>
```

Alles richtig gemacht:

class="heart" ✓

→ CSS-Styling (Hover, Animationen, Farbe)

type="submit" ✓

→ Löst den POST-Request aus (wichtig!)

title="Favorit entfernen" ✓

→ Tooltip für Screenreader + bessere UX

♥ ✓

→ Schönes Emoji-Icon, funktioniert überall

Vollständige Card-Struktur:

```
<article class="card">
  <div class="thumbWrap">
    <a href="{{ url_for('product_details', product_id=p.id) }}">
      
    </a>
  </div>

  <div class="card-body">
    <h3>{{ p.name }}</h3>
    <p>{{ p.price }} €</p>

    <!-- Herz-Button dynamisch gefüllt -->
    <form action="{{ url_for('toggle_favorite', product_id=p.id) }}" method="post" class=
      {% if p in current_user.favorite_products %}
        <button class="heart active" type="submit" title="Favorit entfernen">♥</button>
      {% else %}
        <button class="heart" type="submit" title="Zu Favoriten hinzufügen">♡</button>
      {% endif %}
    </form>
  </div>
</article>
```

CSS-Beispiel:

```
.heart {
  background: none;
  border: none;
  font-size: 24px;
  cursor: pointer;
  padding: 8px;
  border-radius: 50%;
  transition: all 0.2s;
}

.heart:hover {
  transform: scale(1.2);
  background: rgba(255,0,0,0.1);
}

.heart.active {
  color: #ff4757;
}
```

Dein Button ist top! Funktioniert perfekt mit der Toggle-Route. ☐



Erkläre mir die Funktion toggle_favorite Zeile für Zeile.

Hier ist die toggle_favorite Funktion **Zeile für Zeile** erklärt:

```
@app.route("/products/<int:product_id>/toggle_favorite", methods=["POST"])
@login_required
def toggle_favorite(product_id):
    product = Product.query.get_or_404(product_id)

    if product in current_user.favorite_products:
        current_user.favorite_products.remove(product)
        flash("Aus Favoriten entfernt.")
    else:
        current_user.favorite_products.append(product)
        flash("Zu Favoriten hinzugefügt.")

    db.session.commit()
    next_url = request.referrer or url_for("product_list")
    return redirect(next_url)
```

Zeile-für-Zeile Erklärung:

Zeile 1-2: Route-Definition

```
@app.route("/products/<int:product_id>/toggle_favorite", methods=["POST"])
```

- Definiert URL: /products/123/toggle_favorite (123 = product_id)
- <int:product_id> = Parameter aus URL als Integer
- methods=["POST"] = Nur POST-Requests (vom Form-Button)

Zeile 3: Login-Check

```
@login_required
```

- User **muss eingeloggt** sein, sonst Weiterleitung zu Login

Zeile 4: Funktion-Start

```
def toggle_favorite(product_id):
```

- Parameter product_id kommt aus der URL (/products/123/...)

Zeile 5: Produkt laden

```
product = Product.query.get_or_404(product_id)
```

- Holt Produkt mit ID 123 aus Datenbank
- `get_or_404()` = 404-Fehler wenn Produkt nicht existiert

Zeile 6-10: Toggle-Logik

```
if product in current_user.favorite_products:
    current_user.favorite_products.remove(product)
    flash("Aus Favoriten entfernt.")
else:
    current_user.favorite_products.append(product)
    flash("Zu Favoriten hinzugefügt.")
```

- **Prüft:** Ist dieses Produkt schon in den Favoriten des Users?
- **Wenn ja** → `remove(product)` löscht aus Favoriten-Tabelle
- **Wenn nein** → `append(product)` fügt in Favoriten-Tabelle hinzu
- `flash()` = Erfolgsnachricht für nächste Seite

Zeile 11: Datenbank speichern

```
db.session.commit()
```

- **Speichert** alle Änderungen in der Datenbank
- Ohne commit bleiben Änderungen nur im RAM

Zeile 12: Zurück zur vorherigen Seite

```
next_url = request.referrer or url_for("product_list")
```

- `request.referrer` = Von wo kam der User? (Produktliste/Favoriten)
- Fallback: `product_list` wenn referrer leer/undefiniert

Zeile 13: Redirect

```
return redirect(next_url)
```

- Leitet User zur vorherigen Seite weiter
- Seite lädt neu → zeigt aktualisierte Favoriten an

Ablauf-Beispiel:

1. User klickt ♥ Button auf Produkt #123
2. POST-Request → `/products/123/toggle_favorite`
3. Funktion prüft: "123 in User.Favoriten?"
4. Entfernt/Hinzufügt → speichert → redirect
5. User sieht aktualisierten Zustand! ☐

Perfekte Toggle-Logik! Ein Button für hinzufügen **und** entfernen.



Warum wird geprüft: if 'user_id' not in session?

Die Prüfung if 'user_id' not in session: schützt vor **nicht eingeloggten Zugriffen** und verhindert, dass **Gäste** Favoriten hinzufügen/entfernen können.

Warum diese Prüfung notwendig ist:

Flask-Session ohne Flask-Login

Wenn du **kein** flask-login verwendest, speicherst du manuell die User-ID in der Session:

```
# Beim Login:
session['user_id'] = 123  # User ID speichern

# Prüfung vor Favoriten-Aktion:
if 'user_id' not in session:
    flash("Bitte einloggen!")
    return redirect(url_for('login'))

user_id = session['user_id']  # Jetzt sicher nutzbar
```

Mit Flask-Login (besser!)

Du brauchst diese Prüfung NICHT, weil @login_required das übernimmt:

```
@app.route("/products/<int:product_id>/toggle_favorite", methods=["POST"])
@login_required  # ← Ersetzt if 'user_id' not in session
def toggle_favorite(product_id):
    # current_user ist garantiert eingeloggt
    product = Product.query.get_or_404(product_id)
    # ... rest der Funktion
```

Flask-Login macht intern:

1. Prüft session['_user_id'] (Flask-Login speichert so)
2. Lädt User-Objekt via @login_manager.user_loader
3. Setzt current_user auf den echten User
4. Bei ungültiger Session → automatische Weiterleitung zu Login

In deinem Code:

Mit `@login_required` (empfohlen):

```
@login_required  # ✔ Ersetzt manuelle Session-Prüfung
def toggle_favorite(product_id):
    # current_user.favorite_products funktioniert sofort
```

Ohne Flask-Login (manuell):

```
def toggle_favorite(product_id):
    if 'user_id' not in session: # ✘ Manuelle Prüfung nötig
        return redirect(url_for('login'))
    user_id = session['user_id']
    # Komplexere Logik für Favoriten...
```

Fazit:

- `if 'user_id' not in session:` = **manuelle Session-Prüfung** (ohne Flask-Login)
- `@login_required` = **automatische, sichere Prüfung** (mit Flask-Login)
- **Nutze immer Flask-Login** → weniger Code, mehr Sicherheit! ☑

Dein bisheriger Code mit `@login_required` ist perfekt – keine manuelle Session-Prüfung nötig! ☑

✱

was ist session in Flask?

`session` in **Flask** ist wie ein **temporäres Notizbuch für jeden Browser**, das Daten zwischen verschiedenen Seitenaufrufen speichert.

Wie funktioniert's?

```
Browser ↔ Server: HTTP ist "vergesslich" (stateless)
↓
Flask Session: Speichert Daten pro User über mehrere Requests hinweg
```

Technisch:

1. Session = verschlüsseltes Cookie

```
from flask import Flask, session
app = Flask(__name__)
app.secret_key = 'geheim123'  # ← WICHTIG! Verschlüsselt die Session
```

2. Session verhält sich wie Dictionary:

```

@app.route('/set')
def set_session():
    session['username'] = 'max'      # Speichern
    session['level'] = 5             # Mehrere Werte möglich
    return 'Daten gespeichert!'

@app.route('/get')
def get_session():
    if 'username' in session:
        return f'Hallo {session["username"]}' # Auslesen
    return 'Kein User!'

```

3. Wo wird's gespeichert?

```

session['username'] = 'max'
↓
→ Wird zu verschlüsseltem Cookie: eyJ1c2Vyb... (ca. 300 Zeichen)
→ Cookie wird im Browser gespeichert
→ Jeder Request schickt Cookie zurück → Flask entschlüsselt → session verfügbar

```

Praktische Beispiele:

Login-System:

```

@app.route('/login', methods=['POST'])
def login():
    username = request.form['username']
    session['user_id'] = 123      # User merken
    return redirect('/dashboard')

@app.route('/dashboard')
def dashboard():
    if 'user_id' not in session:  # Prüfen ob eingeloggt
        return redirect('/login')
    return 'Willkommen!'

```

Warenkorb:

```

@app.route('/add/<int:product_id>')
def add_to_cart(product_id):
    if 'cart' not in session:
        session['cart'] = []
    session['cart'].append(product_id)
    return 'Produkt hinzugefügt!'

```

In deinem Favoriten-Code:

Mit Flask-Login (@login_required):

```
# session wird intern von Flask-Login verwaltet
# current_user ersetzt manuelle session['user_id'] Prüfung
@login_required
def toggle_favorite(product_id):
    # current_user weiß automatisch, welcher User eingeloggt ist
```

Ohne Flask-Login (manuell):

```
def toggle_favorite(product_id):
    if 'user_id' not in session:      # Manuelle Prüfung!
        return redirect('/login')
    user_id = session['user_id']      # Aus Session lesen
```

Wichtige Punkte:

- ✓ **Vorteil:** Daten persistieren über Seitenwechsel
- ✓ **Sicher:** Verschlüsselt durch SECRET_KEY
- ✗ **Limit:** Cookies haben ~4KB Größenlimit
- ✗ **Serverlast:** Bei vielen Usern (Session auf Server speichern)

Fazit:

```
session = "Erinnere dich an diesen Browser über mehrere Seiten hinweg"
→ Perfekt für Login-Status, Warenkorb, Einstellungen
→ Flask-Login macht das automatisch + sicher für dich!
```

Dein Code mit @login_required nutzt session intern perfekt! 📄



welche Rolle spielt request.referrer bei der Weiterleitung?

`request.referrer` ist der **"Von wo kam der User?"-Hinweis** vom Browser, der bei der Weiterleitung entscheidet, **wohin** der User nach dem Favoriten-Klick zurück soll.

Rolle in deinem Code:

```
next_url = request.referrer or url_for("product_list")
return redirect(next_url)
```

Ablauf Schritt für Schritt:

1. User klickt Herz-Button auf Produktliste

```
Browser: /products ← User war hier
↓ Klick ♥
POST: /products/123/toggle_favorite
↓ toggle_favorite() wird aufgerufen
request.referrer = "/products" ← Browser sagt: "Ich kam von hier"
↓
redirect("/products") ← User landet zurück auf Produktliste
```

2. User klickt Herz-Button auf Favoriten-Seite

```
Browser: /favorites ← User war hier
↓ Klick ♥
POST: /products/123/toggle_favorite
↓
request.referrer = "/favorites" ← Browser sagt: "Ich kam von hier"
↓
redirect("/favorites") ← User landet zurück auf Favoriten
```

3. Wenn `referrer` fehlt/None ist

```
Direct Link / Bookmark / Neues Tab
↓ Klick ♥
POST: /products/123/toggle_favorite
↓
request.referrer = None ← Browser weiß nicht, woher
↓
url_for("product_list") ← Fallback: "/products"
↓
redirect("/products") ← Sichere Standard-Seite
```

Warum `or` so wichtig ist:

```
next_url = request.referrer or url_for("product_list")
```

- **Wahrheitstabelle:**

<code>request.referrer</code>	<code>url_for("product_list")</code>	<code>next_url</code>
<code>"/products"</code>	<code>"/products"</code>	<code>"/products"</code>
<code>None</code>	<code>"/products"</code>	<code>"/products"</code>

`"A or B"` = **A** wenn A existiert, sonst B

Praktische Vorteile:

- ✓ **Intuitive UX:** User bleibt **dort**, wo er war
- ✓ **Funktioniert überall:** Produktliste, Favoriten, Detailseite
- ✓ **Sicherer Fallback:** Nie "nirgendwohin" redirect
- ✓ **Keine Hardcoding:** Flexibel für jede Seite

Alternative (weniger elegant):

```
# Schlecht ✗
if request.referrer:
    return redirect(request.referrer)
return redirect(url_for("product_list"))
```

`request.referrer or url_for("product_list")` = **One-Liner-Perfection!** 📄

Zusammengefasst: `referrer` sagt "wohin zurück?", Fallback sichert ab. User erlebt **nahtlose UX!**



hm

Alles klar! 😊 Hast du noch eine spezifische Frage zur `toggle_favorite`-Funktion oder zum `request.referrer`, oder soll ich zum nächsten Teil übergehen?

ja warum braucht man `db.session.commit()`

`db.session.commit()` ist der **"SPEICHERN"-Button** für die Datenbank – **ohne commit bleiben alle Änderungen nur im RAM!** ^[195]

Warum absolut notwendig:

Was passiert ohne `commit()`:

```
current_user.favorite_products.append(product) # Änderung nur im Python-RAM
# Kein commit()
```

```
RAM: "User favorisiert jetzt Produkt 123" ✓
DB: "User favorisiert immer noch KEIN Produkt 123" ✗
↓ Browser refresh / Neuer Request
RAM weg → Favorit verschwunden!
```

Was passiert MIT `commit()`:

```
current_user.favorite_products.append(product)
db.session.commit()  # ← Änderungen in DB geschrieben!
```

```
RAM: "User favorisiert Produkt 123" ✓
DB: "User favorisiert Produkt 123" ✓
↓ Browser refresh
DB wird neu gelesen → Favorit bleibt!
```

Technischer Ablauf (SQLAlchemy):

1. `append(product)` → Objekt wird "dirty" markiert
2. `session` merkt sich → "Hey, da gibt's was zu speichern"
3. `commit()` → SQLAlchemy:
 - Flush: Schreibt INSERT/UPDATE/DELETE SQL
 - COMMIT: Datenbank transaktion abschließen
4. DB persistent → Änderungen für immer gespeichert

Dein Code ohne `commit()` (kaputt):

```
@app.route("/toggle")
def toggle(product_id):
    product = Product.query.get(product_id)
    current_user.favorite_products.append(product)
    # FEHLER: Kein commit!
    return redirect("/favorites")
```

Ergebnis: Favorit **visuell da**, aber nach **Refresh weg**!

Dein Code mit `commit()` (richtig):

```
@app.route("/toggle")
def toggle(product_id):
    product = Product.query.get(product_id)
    current_user.favorite_products.append(product)
    db.session.commit()  # ✓ Jetzt bleibt's!
    return redirect("/favorites")
```

Memory-Analogie:

```
RAM = Arbeitsmappe (verschwindet wenn PC aus)
DB  = Festplatte  (bleibt für immer)
```

```
append() = Notiz in Arbeitsmappe schreiben
commit() = Notiz auf Festplatte sichern
```

Fazit:

```
append/remove = "Ich plane eine Änderung"  
commit()      = "Änderung jetzt FÜR IMMER speichern!"
```

Ohne commit() = Zuverlässigkeits-Albtraum! 🤖

Deshalb `db.session.commit()` **in JEDER Route mit DB-Änderungen!** 📄

✱

Was passiert ohne `db.session.commit()` in Flask

Ohne `db.session.commit()` passiert **GAR NICHTS** in der Datenbank! 🤖

Was passiert konkret:

1. `current_user.favorite_products.append(product)`
↓
2. Python-RAM: ✔ "Favorit markiert" (nur temporär)
↓
3. KEIN `commit()`
↓
4. Funktion Ende → Response an Browser
↓
5. RAM gelöscht → SQLAlchemy "vergisst" Änderung
↓
6. Datenbank: ✗ UNVERÄNDERT!

Live-Demo (was du siehst vs. Realität):

Browser zeigt: ♥ Produkt in Favoriten (visuell)
↓ Refresh
Browser zeigt: 📄 Produkt NICHT in Favoriten (DB unverändert!)

Warum das passiert:

SQLAlchemy arbeitet in 2 Stufen:

Stufe 1: Python-RAM (staging area)
- `append()`, `remove()` → nur im RAM markiert
- Objekte als "dirty" (geändert) geflaggt

Stufe 2: Datenbank (persistent)
- `commit()` → Flush + SQL COMMIT
- Änderungen wirklich geschrieben

Code-Beispiel (Kaputt):

```
def toggle_favorite(product_id):  
    product = Product.query.get(product_id)  
    current_user.favorite_products.append(product)  # RAM ✓  
    # FEHLER: Kein commit()  
    return redirect('/favorites')
```

Symptom: Favorit **erscheint kurz, verschwindet nach Reload!**

Code-Beispiel (Funktioniert):

```
def toggle_favorite(product_id):  
    product = Product.query.get(product_id)  
    current_user.favorite_products.append(product)  # RAM ✓  
    db.session.commit()                            # DB ✓  
    return redirect('/favorites')
```

Symptom: Favorit **bleibt nach Reload!**

Memory-Analogie:

```
RAM = Excel (vor Speichern)  
DB  = Festplatte (nach Speichern)  
  
append()    = Zelle ändern  
commit()    = Strg+S drücken  
Ohne commit = Excel schließen → Änderung verloren!
```

Häufige Fehlerquellen:

```
✗ Vergessen commit() → "Geht visuell, bleibt nicht"  
✗ commit() vor Query → Transaktionskonflikte  
✓ commit() nach allen Änderungen → Perfekt!
```

Kurz: commit() = **"Änderungen JETZT und FÜR IMMER speichern"**. Ohne = **Phantom-Änderungen!** ☐

✱

Unterschied append(product) **vs** remove(product)

Beide Methoden arbeiten auf der **Many-to-Many-Beziehung** favorite_products, aber **entgegengesetzt!**

1. `append(product)` → **Hinzufügen**

```
current_user.favorite_products.append(product)
```

Was passiert:

```
favorites-Tabelle (DB):  
Vorher: leer für diesen User+Produkt  
Nachher: INSERT user_id=123, product_id=456
```

Effekt: Verknüpfung **NEU** in Zwischentabelle `favorites`

2. `remove(product)` → **Entfernen**

```
current_user.favorite_products.remove(product)
```

Was passiert:

```
favorites-Tabelle (DB):  
Vorher: Zeile user_id=123, product_id=456 existiert  
Nachher: DELETE WHERE user_id=123 AND product_id=456
```

Effekt: Verknüpfung **GELÖSCHT** aus Zwischentabelle

Visueller Vergleich:

```
User 123's favorite_products = [Produkt 1, Produkt 456]
```

1. `append(Produkt 789)`:
 - `[Produkt 1, Produkt 456, Produkt 789]`
 - DB: +1 Zeile in favorites-Tabelle
2. `remove(Produkt 456)`:
 - `[Produkt 1, Produkt 789]`
 - DB: -1 Zeile in favorites-Tabelle

SQLAlchemy-Magie:

Die Beziehung `favorite_products` verhält sich wie eine **Python-Liste**:

```
# Wie bei normalen Python-Listen:  
liste = []  
liste.append(5)    # → [235]  
liste.remove(5)    # → []  
  
# Genau so bei SQLAlchemy-Beziehungen:
```

```
current_user.favorite_products.append(product)    # Fügt DB-Zeile hinzu
current_user.favorite_products.remove(product)    # Löscht DB-Zeile
```

Dein Toggle-Code zusammengefasst:

```
if product in current_user.favorite_products:
    current_user.favorite_products.remove(product) # ✖ Favorit LOESCHEN
    flash("Aus Favoriten entfernt")
else:
    current_user.favorite_products.append(product) # ✔ Favorit HINZUFUEGEN
    flash("Zu Favoriten hinzugefügt")
```

DB-Effekt:

```
favoriten-Tabelle:
user_id | product_id
123     | 456        ← Vorher vorhanden
↓ remove(456)
user_id | product_id
       |           ← Zeile GEGANGEN!

↓ append(789)
user_id | product_id
123     | 789        ← Neue Zeile!
```

Kurz:

- `append()` = "Verknüpfung NEU erstellen" → INSERT
- `remove()` = "Verknüpfung LOESCHEN" → DELETE

Ein Button, zwei Effekte – Toggle-Perfection! ☐

✱

**okay dann so?:if produkt in user.favoriten:
user.favoriten.remove(produkt) else:
user.favoriten.append(produkt)**

JA, GENAU SO! ☐ Perfekt!

```
if produkt in user.favoriten:
    user.favoriten.remove(produkt) # ✔ Entfernen
else:
    user.favoriten.append(produkt) # ✔ Hinzufügen
db.session.commit()
```

Das ist die komplett korrekte Toggle-Logik!

Warum das funktioniert:

1. `if produkt in user.favoriten:`

Prüft die Many-to-Many-Beziehung:

- True → Zeile in favorites-Tabelle existiert
- False → Keine Zeile in favorites-Tabelle

2. `user.favoriten.remove(produkt)`

Löscht die Zeile aus favorites-Tabelle:
`DELETE FROM favorites WHERE user_id=X AND product_id=Y`

3. `user.favoriten.append(produkt)`

Fügt NEUE Zeile in favorites-Tabelle hinzu:
`INSERT INTO favorites (user_id, product_id) VALUES (X, Y)`

Vollständige Funktion:

```
@app.route("/products/<int:product_id>/toggle_favorite", methods=["POST"])
@login_required
def toggle_favorite(product_id):
    produkt = Product.query.get_or_404(product_id)

    if produkt in current_user.favoriten: # ← Deine Schreibweise!
        current_user.favoriten.remove(produkt)
        flash("Aus Favoriten entfernt")
    else:
        current_user.favoriten.append(produkt)
        flash("Zu Favoriten hinzugefügt")

    db.session.commit()
    return redirect(request.referrer or url_for("product_list"))
```

Template passt dazu:

```
<form action="{{ url_for('toggle_favorite', product_id=p.id) }}" method="post">
    {% if p in current_user.favoriten %}
        <button class="heart active" type="submit">♥</button> <!-- Gefüllt -->
    {% else %}
        <button class="heart" type="submit">♡</button> <!-- Leer -->
    {% endif %}
</form>
```

Deine Schreibweise ist 100% richtig!

- produkt statt product → egal (Variablennamen frei wählbar)
- user.favoriten statt current_user.favorite_products → egal (Beziehungsname aus Model)

Ein-Klick-Toggle = perfekt implementiert! ☐



Brauche ich noch db.session.commit() nach remove/append

JA, db.session.commit() ist **ABSOLUT NOTWENDIG** nach remove() **und** append()! ☐

Ohne commit() passiert NICHTS in der Datenbank:

```
if produkt in user.favoriten:
    user.favoriten.remove(produkt)    # RAM: ✔ "Gelöscht"
else:
    user.favoriten.append(produkt)    # RAM: ✔ "Hinzugefügt"
# KEIN commit()
return redirect('/favorites')
```

Ergebnis:

```
Browser: Favoritstatus wechselt (visuell) ✔
↓ Refresh
Browser: Favoritstatus WIE GEVORHER! ✕
```

Warum commit() zwingend nötig:

1. append()/remove() → Nur RAM-Änderung
2. SQLAlchemy merkt: "Änderung pending"
3. Funktion endet → Response an Browser
4. RAM gelöscht → SQLAlchemy "vergisst" alles
5. DB unverändert → Nächste Query liest alten Zustand

Korrekte Reihenfolge:

```
if produkt in user.favoriten:
    user.favoriten.remove(produkt)
else:
    user.favoriten.append(produkt)

db.session.commit()  # ← OHNE DAS FUNKTIONIERT ES NICHT!
```

Memory-Analogie:

append/remove = Notiz auf Zettel schreiben (RAM)
commit() = Zettel in Schublade abheften (DB)

Ohne commit() = Zettel im Papierkorb! ☹

Vollständige Funktion (muss so sein):

```
@app.route("/products/<int:product_id>/toggle_favorite", methods=["POST"])
@login_required
def toggle_favorite(product_id):
    produkt = Product.query.get_or_404(product_id)

    if produkt in current_user.favoriten:
        current_user.favoriten.remove(produkt)
        flash("Aus Favoriten entfernt")
    else:
        current_user.favoriten.append(produkt)
        flash("Zu Favoriten hinzugefügt")

    db.session.commit() # ← ZWINGEND NOTWENDIG!

    return redirect(request.referrer or url_for("product_list"))
```

Kurz: append/remove = **Plan ändern**, commit() = **Plan ausführen**! Ohne commit bleibt alles nur **Phantom-Änderung**! 🤖

✱

Welche Tabellen entstehen in der DB durch diese favoriten funktion?

Durch die Favoriten-Funktion entstehen **3 Tabellen** in der Datenbank:

1. `user` Tabelle (besteht schon)

```
user:
| id | email          |
|----|-----|
| 1  | max@skincare.de |
| 2  | anna@test.de   |
```

2. product Tabelle (besteht schon)

```
product:
| id | name           | brand           | skin_type | price |
|----|-----|-----|-----|-----|
| 1  | Hydra Serum    | The Ordinary    | dry       | 24.99 |
| 2  | Retinol Cream  | CeraVe          | oily      | 19.99 |
```

3. favorites Tabelle (NEU! Automatisch durch Many-to-Many)

```
favorites: ← Zwischentabelle für Verknüpfungen
| user_id | product_id |
|-----|-----|
| 1        | 1          | ← Max favorisiert Hydra Serum
| 1        | 2          | ← Max favorisiert Retinol Cream
| 2        | 1          | ← Anna favorisiert auch Hydra Serum
```

Wie entsteht die favorites Tabelle?

Durch dein Model:

```
favorites = db.Table(
    "favorites",          # ← Tabellenname
    db.Column("user_id", db.Integer, db.ForeignKey("user.id"), primary_key=True),
    db.Column("product_id", db.Integer, db.ForeignKey("product.id"), primary_key=True),
)
```

SQLAlchemy erstellt automatisch:

```
CREATE TABLE favorites (
    user_id INTEGER REFERENCES user(id),
    product_id INTEGER REFERENCES product(id),
    PRIMARY KEY (user_id, product_id)
);
```

Was passiert bei append() / remove()?

```
current_user.favoriten.append(produkt_1)
↓
INSERT INTO favorites (user_id, product_id) VALUES (1, 1)

current_user.favoriten.remove(produkt_1)
↓
DELETE FROM favorites WHERE user_id=1 AND product_id=1
```

Abfrage Favoriten (`current_user.favoriten.all()`):

```
SELECT product.* FROM product
JOIN favorites ON product.id = favorites.product_id
WHERE favorites.user_id = 1;
```

Zusammenfassung:

3 Tabellen insgesamt:

- ✓ user ← Benutzerdaten
- ✓ product ← Produktdaten
- favorites ← VERKNÜPFUNG (neu durch Favoriten!)

Die favorites Tabelle ist das Herzstück – sie speichert **welcher User welches Produkt favorisiert!** □



1. <https://stackoverflow.com/questions/68432703/how-to-show-a-signed-in-users-list-of-favorites>
2. <https://stackoverflow.com/questions/59905686/best-practices-for-favorites-feature-in-app>
3. <https://stackoverflow.com/questions/61415988/flask-storing-get-and-update-list-in-session-using-flask-session>
4. <https://www.holdapp.com/blog/beauty-ecommerce-apps-feature-ideas>
5. <https://swankyagency.com/ux-strategies-skincare-ecommerce-website/>
6. https://www.reddit.com/r/flask/comments/h7l6mp/bookmark_feature_in_a_flask_webapp/
7. <https://www.appsloveworld.com/coding/python3x/235/flask-storing-get-and-update-list-in-session-using-flask-session>
8. <https://blog.teclado.com/how-to-add-user-logins-to-your-flask-website/>
9. https://www.reddit.com/r/flask/comments/kv762b/flask_user_management/
10. <https://testdriven.io/blog/flask-sessions/>
11. <https://www.rustcodeweb.com/2025/05/flask-defining-relationships-in-models.html>
12. <https://michaelcho.me/article/many-to-many-relationships-in-sqlalchemy-models-flask/>
13. <https://www.digitalocean.com/community/tutorials/how-to-use-templates-in-a-flask-application>
14. <https://stackoverflow.com/questions/70943354/python-flask-how-to-display-list-of-items-in-html-template>
15. <https://www.digitalocean.com/community/tutorials/how-to-use-one-to-many-database-relationships-with-flask-sqlalchemy>
16. <https://stackoverflow.com/questions/25148536/complex-many-to-many-query-with-flask-sqlalchemy>
17. <https://stackoverflow.com/questions/22110515/flask-help-understanding-primaryjoin-secondaryjoin-on-a-many-to-many-relationships>
18. https://www.reddit.com/r/flask/comments/qasepy/query_using_flask_sqlalchemy_manytomany/
19. https://www.youtube.com/watch?v=CBXH_iRKFdI
20. <https://dev.to/freddiemazzilli/flask-sqlalchemy-relationships-exploring-relationship-associations-igo>

21. <https://stackoverflow.com/questions/68432703/how-to-show-a-signed-in-users-list-of-favorites>
22. <https://stackoverflow.com/questions/33372714/removing-adding-an-object-from-a-collection-in-sqlalchemy>
23. <https://calmops.com/programming/python/flask-fundamentals-routing-templates-forms/>
24. <https://stackoverflow.com/questions/46270131/adding-the-like-feature-to-posts-and-comments-in-a-flask-web-application>
25. <http://s.90done.com/post/116>
26. <https://stackoverflow.com/questions/70943354/python-flask-how-to-display-list-of-items-in-html-template>
27. <https://stackoverflow.com/questions/66084995/how-to-create-dynamic-route-in-flask-after-form-submit>
28. https://www.reddit.com/r/flask/comments/prww3l/append_will_append_to_a_relationship_attribute_of/
29. https://www.reddit.com/r/flask/comments/fqa4nf/what_are_the_most_advanced_features_you_can/
30. <https://dev.to/jessicagarson/plotting-bookmarks-with-flask-matplotlib-and-oauth-20-44g1>
31. <https://www.youtube.com/watch?v=-8gXGbXPjBE>
32. <https://www.youtube.com/watch?v=HmjSFERXaKw>
33. <https://www.python-forum.de/viewtopic.php?t=54685>
34. <https://www.datacamp.com/de/tutorial/sqlalchemy-tutorial-examples>
35. <https://www.digitalocean.com/community/tutorials/processing-incoming-request-data-in-flask-de>
36. <https://calmops.com/programming/python/flask-fundamentals-routing-templates-forms/>
37. <https://www.digitalocean.com/community/tutorials/how-to-use-templates-in-a-flask-application>
38. <https://stackoverflow.com/questions/70943354/python-flask-how-to-display-list-of-items-in-html-template>
39. <https://www.python-forum.de/viewtopic.php?t=54727>
40. <https://www.youtube.com/watch?v=ZgwHvNCn3QQ>
41. https://www.reddit.com/r/flask/comments/xwhvic/how_to_use_flask_with_existing_database/
42. https://www.reddit.com/r/flask/comments/g7sv39/what_is_the_proper_way_to_connect_to_the_database/
43. <https://renaud-detry.net/teaching/flask/tutorial/database.html>
44. <https://info.blaisepascal.fr/web-flask-base-de-donnees/>
45. <https://fintechpython.pages.oit.duke.edu/jupyternotebooks/7-WebDevelopment/13-flask-layout.html>
46. <https://www.geeksforgeeks.org/python/flask-app-routing/>
47. <https://www.python-forum.de/viewtopic.php?t=54685>
48. <https://www.digitalocean.com/community/tutorials/processing-incoming-request-data-in-flask-de>
49. <https://www.datacamp.com/de/tutorial/sqlalchemy-tutorial-examples>
50. <https://www.digitalocean.com/community/tutorials/how-to-structure-a-large-flask-application-with-flask-blueprints-and-flask-sqlalchemy>
51. <https://stackoverflow.com/questions/14415500/common-folder-file-structure-in-flask-app>
52. https://www.reddit.com/r/flask/comments/12c8hf1/flask_app_folder_structure_suggestions/
53. https://www.reddit.com/r/flask/comments/13kisfa/routes_in_flask/

54. https://www.reddit.com/r/flask/comments/e1fz7o/how_to_structure_flask_application/
55. <https://www.youtube.com/watch?v=lq7pC1rITxk>
56. <https://stackoverflow.com/questions/14415500/common-folder-file-structure-in-flask-app/49815759>
57. https://www.reddit.com/r/flask/comments/krohx5/can_someone_please_explain_to_me_how_the_flask/
58. <http://ttl255.com/jinja2-tutorial-part-2-loops-and-conditionals/>
59. <https://pushmetrics.io/learn/jinja/jinja-loops/>
60. <https://www.digitalocean.com/community/tutorials/processing-incoming-request-data-in-flask-de>
61. <https://calmops.com/programming/python/flask-fundamentals-routing-templates-forms/>
62. <https://doc.qt.io/Qt/5.12/template-syntax.html>
63. <https://stackoverflow.com/questions/12145434/how-to-output-loop-counter-in-python-jinja-template>
64. <https://doc.qt.io/Qt/5.12/template-syntax.html>
65. <https://gist.github.com/kevin-brown/ec945b60024d35ed9c09ec70142af7fe>
66. <https://stackoverflow.com/questions/29706099/how-to-make-a-for-loop-in-jinja>
67. <https://jinja.palletsprojects.com/en/stable/templates/>
68. <https://blog.datamix.com/templating-mit-jinja2-der-schnelleinstieg-e631a54aff35>
69. <https://documentation.bloomreach.com/engagement/docs/jinja-syntax>
70. <https://fintechpython.pages.oit.duke.edu/jupyternotebooks/7-WebDevelopment/14-flask-template.html>
71. <https://python-adv-web-apps.readthedocs.io/en/latest/flask3.html>
72. <https://www.youtube.com/watch?v=IhpNRDGzInw>
73. <https://stackoverflow.com/questions/54015800/flask-bootstrap-cards-and-jinja-templating>
74. <https://stackoverflow.com/questions/24847753/flask-jinja2-how-to-separate-header-base-and-footer>
75. <https://www.geeksforgeeks.org/python/templating-with-jinja2-in-flask/>
76. <https://flask.palletsprojects.com/en/stable/tutorial/templates/>
77. <https://stackoverflow.com/questions/72235676/best-practices-in-structuring-jinja-templates-flask>
78. <https://www.codecademy.com/learn/learn-flask/modules/flask-templates-and-forms/cheatsheet>
79. <https://realpython.com/primer-on-jinja-templating/>
80. <https://dev.to/fermyon/part-3-templating-html-with-python-jinja2-and-serverless-webassembly-53fa>
81. <https://jinja.palletsprojects.com/en/stable/templates/>
82. <https://python.code-maven.com/minimal-example-generating-html-with-python-jinja>
83. <https://stackoverflow.com/questions/34305150/html-templates-using-jinja2>
84. https://python-web.teclado.com/section07/lectures/05_jinja2_data_structures/
85. <https://documentation.bloomreach.com/engagement/docs/datastructures>
86. <https://stackoverflow.com/questions/72235676/best-practices-in-structuring-jinja-templates-flask>
87. <https://www.geeksforgeeks.org/python/getting-started-with-jinja-template/>
88. <https://codesignal.com/learn/courses/getting-started-with-flask-and-web-development/lessons/displaying-dynamic-content-with-jinja2-templates>
89. <https://blog.datamix.com/templating-mit-jinja2-der-schnelleinstieg-e631a54aff35>
90. <https://jinja.palletsprojects.com/en/stable/templates/>
91. <https://www.python-forum.de/viewtopic.php?t=55310>

92. https://www.reddit.com/r/homeassistant/comments/1cku7yo/yaml_jinja_templating_course_episode_3_ifthen/
93. https://www.reddit.com/r/dataengineering/comments/yll0e8/dbt_jinja_question_around_seeds_and_loops/
94. <https://dvmn.org/encyclopedia/modules/jinja2-operators/>
95. <https://stackoverflow.com/questions/12655155/jinja2-for-loop-with-conditions>
96. https://www.reddit.com/r/flask/comments/yj1ukg/jinja_if_statements_not_working_for_me/?tl=fr
97. <https://docs-python.ru/packages/modul-jinja2-python/tsikly-for-in-shablonah-jinja2/>
98. <https://www.registrarcorp.com/de/blog/cosmetics-de/cosmetic-regulations-de/cosing-database/>
99. <https://v0.app/chat/skincare-database-design-AtEECGAeNYi>
100. <https://www.glynt.com/professionals/service/inhaltsstoffdatenbank>
101. <https://www.gesundheitsdienstportal.de/files/Kosmetika-Inhaltsstoffe-Funktionen.pdf>
102. <https://www.haut.de/inhaltsstoffe-inci/>
103. https://www.reddit.com/r/webdev/comments/rt6m34/how_can_i_build_a_comprehensive_database_of/
104. https://www.reddit.com/r/datasets/comments/grf1s4/looking_for_a_skincare_data_set_that_includes/
105. <https://www.kaggle.com/datasets/muhammadsrefki/dataset-skincare>
106. <https://www.kaggle.com/eward96/skincare-products-clean-dataset/activity>
107. https://runebook.dev/en/articles/flask/api/index/flask.url_for
108. <https://ask42.us/course/flask-introduction/chapter/url/>
109. <https://stackoverflow.com/questions/7478366/create-dynamic-urls-in-flask-with-url-for>
110. <https://www.sitepoint.com/flask-url-routing/>
111. https://www.geeksforgeeks.org/python/flask-url-helper-function-flask-url_for/
112. https://www.reddit.com/r/flask/comments/awuu1q/handling_url_for_parameters_with_flask/
113. <https://www.youtube.com/watch?v=7X1X39vccW4>
114. <https://flask.palletsprojects.com/en/stable/api/>
115. <https://cs.wellesley.edu/~cs304flask/readings/flask2/flask2.html>
116. <https://www.youtube.com/watch?v=mXORRTVhNwU>
117. <https://www.youtube.com/watch?v=TIPfLxtPmbg>
118. <https://www.youtube.com/watch?v=85cgc4myuDY>
119. <https://wings2pc.tistory.com/entry/웹-애플로그래밍-Jinja2-extends-사용해서-html-나누기layout>
120. <https://stackoverflow.com/questions/28295453/structuring-jinja-flask-templates-with-multiple-child-templates>
121. <https://flask.palletsprojects.com/en/stable/patterns/templateinheritance/>
122. <https://www3.tuhh.de/itbh/einfuehrung-in-die-informationstechnik/einheiten/webanwendungen-mit-flask/das-template-system-von-flask/>
123. <https://www.youtube.com/watch?v=u04VF5vWvpl>
124. <https://blog.datamix.com/templating-mit-jinja2-der-schnelleinstieg-e631a54aff35>
125. <https://testdriven.io/tips/b94ad5b8-3238-40e6-87ec-6e31086faabe/>
126. <https://flask.website/flask/jinja>

127. <https://vibecodingwithfred.com/tutorials/shopping-flask/>

128. <https://www.codecademy.com/learn/learn-flask/modules/flask-templates-and-forms/cheatsheet>

129. <https://stackoverflow.com/questions/54015800/flask-bootstrap-cards-and-jinja-templating>

130. <https://flask.blog/flask-render-jinja-templates>

131. <https://flask.palletsprojects.com/en/stable/tutorial/templates/>

132. <https://stackoverflow.com/questions/72235676/best-practices-in-structuring-jinja-templates-flask>

133. <https://www.codementor.io/@chirilovadrian360/jinja-template-short-introduction-and-free-samples-1assoixldd>

134. <https://dev.to/sm0ke/jinja-templates-curated-list-3m06>

135. <https://flask.palletsprojects.com/en/stable/templating/>

136. <https://www.codecademy.com/learn/introduction-to-flask/modules/flask-templates-and-forms/cheatsheet>

137. <https://stackoverflow.com/questions/58052653/all-links-in-python-flask-form-behave-like-submit>

138. <https://stackoverflow.com/questions/19794695/flask-python-buttons>

139. <https://stackoverflow.com/questions/72148700/flask-select-which-form-to-post-by-button-click>

140. https://www.reddit.com/r/flask/comments/b21bv7/multiple_forms_and_buttons_best_practices/?tl=fr

141. https://python-adv-web-apps.readthedocs.io/en/latest/flask_forms.html

142. https://frhyme.github.io/python-lib/a_flask_with_button/

143. https://www.reddit.com/r/flask/comments/b21bv7/multiple_forms_and_buttons_best_practices/

144. <https://www.geeksforgeeks.org/python/create-a-bookmark-organizer-for-website-using-flask/>

145. https://www.reddit.com/r/flask/comments/qbi78b/how_to_make_a_button_on_a_page_modify_the_database/

146. <https://experienceleague.adobe.com/de/docs/dynamic-media-developer-resources/library/viewers-aeam-assets-dmc/ecatalog/customizing-ecatalog/r-html5-ecatalog-viewer-20-customize-removefavorite>

147. <https://www.html.de/threads/formular-und-button.23455/>

148. <https://www-user.tu-chemnitz.de/~lfe/selfhtml61/tchg.htm>

149. <https://developer.mozilla.org/de/docs/Web/HTML/Reference/Elements/button>

150. <https://stackoverflow.com/questions/27790066/html-adding-and-removing-to-favorites>

151. <http://euklid.mi.uni-koeln.de/c/doc/html/selfhtml-7.0/tchk.htm>

152. <https://wiki.selfhtml.org/wiki/Checkbox-Hack>

153. <https://stackoverflow.com/questions/2460100/remove-the-complete-styling-of-an-html-button-submit>

154. https://developer.mozilla.org/de/docs/Learn_web_development/Core/Structuring_content/HTML_forms

155. <https://www.html.de/threads/wie-aender-ich-den-button-im-senden-feld-eines-formulars.31870/>

156. https://www.reddit.com/r/flask/comments/117ieup/seeking_opinion_re_flaskjavascript_interfacing/

157. <https://www.youtube.com/watch?v=45P3xQPAYxc>

158. <https://www.youtube.com/watch?v=YTb05mkRxwM>

159. <https://stackoverflow.com/questions/51057966/flask-toggle-button-with-dynamic-label>

160. https://python-adv-web-apps.readthedocs.io/en/latest/flask_forms.html

161. <https://flask.palletsprojects.com/en/stable/quickstart/>

162. <https://flask.palletsprojects.com/en/stable/api/>

163. https://www.reddit.com/r/flask/comments/n5q3wi/how_does_flask_know_what_view_function_to_execute/

164. <https://flask.palletsprojects.com/en/stable/config/>

165. <https://flask.palletsprojects.com/en/stable/tutorial/templates/>

166. <https://stackoverflow.com/questions/15156132/flask-login-how-to-get-session-id>

167. <https://flask-login.readthedocs.io/en/0.6.3/>

168. <https://flask-login.readthedocs.io>

169. <https://stackoverflow.com/questions/43473960/check-flask-cookie-for-user-id-and-obtain-user-if-it-exists-or-redirect-if-not>

170. <https://www.pythontutorials.net/blog/check-if-user-is-logged-in-with-flask-login-in-template/>

171. <https://www.digitalocean.com/community/tutorials/how-to-add-authentication-to-your-app-with-flask-login-de>

172. <https://www.maxlist.xyz/2019/06/29/flask-google-login-check/>

173. <https://stackoverflow.com/questions/41721927/how-does-flask-login-identify-different-users-that-have-logged-in-by-a-single-session>

174. <https://github.com/maxcountryman/flask-login/issues/306>

175. <https://inf.zone/lectures/9-flask/short-9-3-sessions/>

176. <https://www.youtube.com/watch?v=b21hkj9bsbM>

177. <https://pythongeeks.org/flask-session/>

178. <https://pythonbasics.org/flask-sessions/>

179. <https://www3.tuhh.de/itbh/informatik-202021/veranstaltungsskript/python/flask/die-vergesslichkeit-des-servers-sessions/>

180. <https://kimjingo.tistory.com/96>

181. <https://testdriven.io/blog/flask-sessions/>

182. https://www.reddit.com/r/flask/comments/164jgrh/what_are_sessions_how_to_use_sessions_in_flask/

183. <https://www.python-forum.de/viewtopic.php?t=47445>

184. <https://www.youtube.com/watch?v=GgS8-mn9zoM>

185. <https://stackoverflow.com/questions/61756316/flask-request-referrer-returns-none-when-referred-from-external-url>

186. <https://insource-mkd.co.jp/staff-blog/14278/>

187. https://www.reddit.com/r/flask/comments/fo71ie/requestreferrer_is_not_redirecting_to_previous_url/

188. <https://www.delftstack.com/de/howto/python-flask/flask-redirect/>

189. <https://tedboy.github.io/flask/generated/generated/flask.Request.referrer.html>

190. <https://www.python-forum.de/viewtopic.php?t=58301>

191. <https://stackoverflow.com/questions/54580323/flask-get-request-referrer-endpoint>

192. <https://stackoverflow.com/questions/14343812/redirecting-to-url-in-flask>

193. <https://www.geeksforgeeks.org/python/redirecting-to-url-in-flask/>

194. <https://flask.palletsprojects.com/en/stable/api/>

195. <https://flask-sqlalchemy.readthedocs.io/en/stable/queries/>

196. <https://www.francoisvoron.com/blog/sqlalchemy-stop-calling-session-commit>

197. <https://stackoverflow.com/questions/62816498/sqlalchemy-with-flask-db-session-commit-not-working>

198. <https://github.com/miguelgrinberg/flasky/issues/483>

199. <https://stackoverflow.com/questions/63403999/python-flask-sqlalchemy-do-i-have-to-commit-session-after-a-query>

200. https://www.reddit.com/r/flask/comments/jq0say/dbsessionaddx_and_dbsessioncommit/

201. <https://flask.palletsprojects.com/en/stable/patterns/sqlalchemy/>

202. <https://leapcell.io/blog/de/aufbau-robuster-anwendungen-mit-flask-sqlalchemy-modellen-beziehungen-und-transaktionsverwaltung>

203. https://www.reddit.com/r/flask/comments/18sg4zv/is_dbsessionadd_and_dbsessioncommit_outdated/

204. <https://www.francoisvoron.com/blog/sqlalchemy-stop-calling-session-commit>

205. https://www.reddit.com/r/flask/comments/jq0say/dbsessionaddx_and_dbsessioncommit/?tl=es-419

206. <https://stackoverflow.com/questions/62816498/sqlalchemy-with-flask-db-session-commit-not-working>

207. <https://blog.csdn.net/miyagiSimple/article/details/121213629>

208. <https://stackoverflow.com/questions/44903485/does-db-session-commit-change-app-context-in-flask-sqlalchemy>

209. <https://github.com/miguelgrinberg/flasky/issues/483>

210. <https://flask-sqlalchemy.readthedocs.io/en/stable/queries/>

211. https://www.reddit.com/r/flask/comments/18sg4zv/is_dbsessionadd_and_dbsessioncommit_outdated/

212. https://www.reddit.com/r/flask/comments/lzl30j/cannot_commit_to_database_with_flask_sqlalchemy/

213. https://www.reddit.com/r/flask/comments/jq5sy7/dbsessioncommit_not_working_inside_instance/

214. https://blog.csdn.net/weixin_38644677/article/details/88378079

215. <https://stackoverflow.com/questions/33372714/removing-adding-an-object-from-a-collection-in-sqlalchemy>

216. <https://www.youtube.com/watch?v=qY8HfI8vMFE>

217. <https://github.com/sqlalchemy/sqlalchemy/discussions/9081>

218. <https://deepnote.com/blog/ultimate-guide-to-sqlalchemy-library-in-python>

219. https://www.reddit.com/r/flask/comments/prww3l/append_will_append_to_a_relationship_attribute_of/

220. https://www.sqlalchemy.org/docs/14/changelog/migration_14.html

221. https://www.reddit.com/r/PostgreSQL/comments/ksc6zk/how_to_sessionappend_without_creating_a_new/

222. https://www.reddit.com/r/learnpython/comments/1g3trbe/i_am_learning_sqlalchemy_and_im_not_sure_if_i_am/

223. <https://stackoverflow.com/questions/49266409/button-click-in-html-to-update-sqlalchemy-db-in-flask>

224. <https://auth0.com/blog/sqlalchemy-orm-tutorial-for-python-developers/>

225. <https://deepnote.com/blog/ultimate-guide-to-sqlalchemy-library-in-python>

226. https://www.youtube.com/watch?v=r6XasrHGI_A

227. <https://en.wikipedia.org/wiki/SQLAlchemy>

228. https://www.reddit.com/r/flask/comments/qpiukb/sqlalchemy_delete_not_working/

229. <https://www.python-forum.de/viewtopic.php?t=40902>

230. <https://stackoverflow.com/questions/60807495/is-there-any-order-for-add-versus-delete-when-committing-in-sqlalchemy>
231. <https://coderpad.io/blog/development/understanding-transactions-in-sqlalchemy/>
232. <https://flask-sqlalchemy.readthedocs.io/en/stable/queries/>
233. <https://stackoverflow.com/questions/14665834/when-should-i-commit-with-sqlalchemy-using-a-for-loop>
234. <https://github.com/sqlalchemy/sqlalchemy/discussions/9566>
235. <https://github.com/sqlalchemy/sqlalchemy/discussions/7483>
236. <https://stackoverflow.com/questions/60691513/sqlalchemy-how-to-delete-and-flush-instead-of-commit>
237. <https://stackoverflow.com/questions/5756559/how-to-build-many-to-many-relations-using-sqlalchemy-a-good-example/23424290>
238. <https://stackoverflow.com/questions/76509491/how-do-i-build-a-sqlalchemy-many-to-many-relationship-on-a-single-table-with-ext>
239. https://www.reddit.com/r/learnpython/comments/22d1bc/manytomany_relationship_in_sqlalchemy/
240. <https://www.digitalocean.com/community/tutorials/how-to-use-many-to-many-database-relationships-with-flask-sqlalchemy>
241. <https://www.python-forum.de/viewtopic.php?t=54091>
242. <https://sqlmodel.tiangolo.com/tutorial/many-to-many/>
243. <https://www.youtube.com/watch?v=3IDiEC0ZwPg>
244. https://www.youtube.com/watch?v=iosh_DWnliE
245. <https://github.com/sqlalchemy/sqlalchemy/discussions/8789>